2026-5-8 C++string--底层实现

- 1.默认成员函数
  - 1.1 构造函数
    - 1.1.1 无参构造
    - 1.1.2 带参构造
    - 1.1.3 将二者合并
  - 1.2 析构函数
  - 1.3拷贝构造
  - 1.4 运算符重载
    - 1.4.1 oparetor[ ]下标访问运算符重载
    - 1.4.2 oparetor操纵符重载
    - 1.4.3 oparetor!=操纵符重载
    - 1.4.4 oparetor+=操纵符重载
    - 1.4.5 比较运算符重载
    - 1.4.6 流插入运算符
    - 1.4.7流输出运算符
  - 1.5 赋值运算符重载
- 2. 修改器
  - 2.1swap交换函数
  - 2.2 push_back()尾插函数
  - 2.3 pop_back去掉尾部元素
  - 2.4 insert插入函数
  - 2.5 erase修改函数
  - 2.5 clear清除
- 3. 容量和空间相关
  - 3.1reserve空间开辟函数
  - 3.2 append 追加函数
  - 3.3 size返回大小
- 4.查找和提取
  - 4.1c_str返回常量指针
  - 4.2 find查找函数
  - 4.3 substr提取某一段元素
- 5.迭代器

1.默认成员函数

1.1 构造函数

1.1.1 无参构造

头文件中对构造函数的初始化声明如下：

```
//如果我们按照如下方式在对构造函数初始化会出现一些问题
string::string()//不带参数的构造
    : _str(nullptr)
    ,_size(0)
    ,_capacity(0){
}
```

如果我们在构造函数中将 `_str` 初始化为 `nullptr`，那么当调用 `c_str()` 或使用 `cout` 输出字符串时，`cout` 会对 `const char*` 类型的指针按字符串内容进行打印，而不是打印地址。此时遇到空指针，就会发生解引用空指针的行为，导致程序崩溃。
因此，即使表示空字符串，也应当让 `_str` 指向一块有效内存（例如 `new char[1]{'\0'}`），避免空指针解引用。
这里为什么说在C++中`const char*`与`char*`是例外?
我们举个例子来看看：

```
int a = 10;
int* p = &a;
cout << p << endl;       // 输出地址，如 0x7ffd1234

const char* str = "hello";
cout << str << endl;     // 输出 hello（不是地址）
//但在面试中下述方式在面试中可能是错误的
const char* p = nullptr;
cout << p << endl;    // ❌ 未定义行为（大概率崩溃）
//在这里编译器会按照“打印字符串”的逻辑去处理他会试图从nullptr
//开始读取直到\0但是空指针解引用直到崩溃
```

但如果你要用`const char*`有没有什么办法打印地址？强制类型转化为void*。

```
const char* str = "hello";
cout << (void*)str << endl;    // 打印地址
```

构造函数的正确写法如下：

```
//无参默认构造的正确写法
string::string()//不带参数的构造
//必须要先开一个空间，先存入一个终止
    : _str(new char[1]{'\0'})
    ,_size(0)
    ,_capacity(0){
}
```

1.1.2 带参构造

很多人在一开始写带参构造的时候会这样写：

```
string::string(const char* str)
    //:_str(str)这样直接改str是不对的,不能用这个来直接初始化
    //const char*涉及到权限放大的问题，但是声明的时候是不能加const的
    //因为我们指明了string是可以修改的
    :_str(new char[strlen(str)+1])//加一留给'\0'
    , _size(strlen(str))
    , _capacity(strlen(str)) {
    //将数据拷贝过来
    strcpy(_str, str);//将str拷贝给_str,包括'\0'
}
```

但是这个东西有一些缺陷，问题在于strlen这个东西，strlen是在运行的时候作用的，sizeof是在编译的时候作用的，strlen用的太多了不好。
为什么strlen用太多的不好？我们来看看strlen底层是怎样实现的？

```
size_t my_strlen(const char* s) {
    size_t len = 0;
    while (s[len] != '\0') {//逐字节扫描是否有‘\0’看有多少个字符
        ++len;
    }
    return len;
}
```

那下述写法正确吗？

```
string::string(const char* str)
    : _size(strlen(str))
    ,_str(new char[_size + 1])//加一留给'\0'
```

```
    , _capacity(_size) {
    //将数据拷贝过来
    strcpy(_str, str);//将str拷贝给_str,包括'\0'
}
```

通过之前的知识我们学过，初始化列表初始化的顺序要是声明的顺序，那前面声明部分就要改为：

```
private:
    size_t _size=0;
    char* _str = nullptr;
    size_t _capacity=0;
```

但是这种代码就很不稳定（耦合的代码），如果你的代码交给了别人来维护，别人觉得这种写法不是很合常理，顺序一交换程序不久挂了吗？所以说这种解决方式实际上是不合理的。那我们可以不用初始化列表来初始化。（上面哪个没屏蔽掉程序也不一定会挂，上述程序会不会挂取决于后初始化的成员会不会被先使用，但是该程序没挂这个程序也是一个未定义行为）越界不一定会报错，不一定能检查出来。

```
string::string(const char* str)
    : _size(strlen(str)){
    _str = (new char[_size + 1]);
    _capacity = (_size);//只有引用，const，没有初始化构造的值才必须使用初始化列表
    strcpy(_str, str);
}
```

`strcpy(_str, str);`为什么不能用交换？交换不能完成拷贝，不如直接指向str，但是str的数据是在常量区的，常量区的数据是不能修改的，但是我们_str里面的内容是要修改的。要去堆上开辟一块内存给拷贝过来进行操作，所以这里不能使用交换。

1.1.3 将二者合并

我们说带参数构造和无参构造二者可以写成一个：那么怎么合并呢？**写全缺省**
首先在声明中：strlen会对_str解引用，不能不用strlen

```
string::string(const char* str=nullptr);
```

常量串默认有"\0"这里strlen是更具\0判别的，这里会导致你构造的字符串在一开始就有了一个\0这实际上是无法判别的。

```
string::string(const char* str="\0"); //这里会导致当你在构造一个空出串的时候实际的内容变\0\0，\0在这里就变成了一个显式的字面量
```

所以正确的写法是：

```
string::string(const char* str=" "); //啥也没有，一看就是空的
```

综合上述，我们合并后的代码为：

```
string::string(const char* st
    : _size(strlen(str)){
    _str = (new char[_size + 1]);
    _capacity = (_size);//只有引用，const，没有初始化构造的值才必须使用初始化列表
    //strcpy(_str, str);
    memcpy(_str, str,_size+1);
}
```

为什么推荐使用`memcpy`？因为strcpy是根据`\0`计数的，`memcpy`是啥也不管的赋值，如果我们原本的串中间就是有一个`\0`，使用memcpy这样在拷贝的时候就不会终止了。

1.2 析构函数

释放空间并置空：

```
//~string();
string::~string() {
    delete[] _str;
    _str = nullptr;
    _capacity = _size = 0;
}
```

1.3拷贝构造

```
/*string::string(const string& s) {
    _str = new char[s._capacity + 1];
    memcpy(_str, s._str, s._size + 1);
    _size = s._size;
    _capacity = s._capacity;
}//借助别人完成拷贝*/
string::string(const string& s) {
    string tmp(s._str);//这里是调用普通构造
    swap(tmp);
}
```

`swap(tmp)` 交换的是当前对象 `*this` 和临时对象 `tmp` 的内部成员（指针、大小、容量），不是交换 `this` 指针本身。这样做的目的是把已构造好的临时对象的资源“偷”过来，让当前对象成为有效对象，而临时对象在析构时会释放原先的垃圾资源。

1.4 运算符重载

1.4.1 `oparetor[ ]` 下标访问运算符重载

```
```C++
//char& operator[](size_t i);//[]操作，这个是可读可写的
char& string::operator[](size_t i) {
    //首先断言检查i是否合法
    assert(i < _size);
    return _str[i];
}
//const char& operator[](size_t i)const;//这个是只能读的
const char& string::operator[](size_t i)const {
    assert(i < _size);
    return _str[i];
}
```

运算符的重载就能让我们对其进行操作：

```
void sting_text01() {
    string s1;
    cout << s1.c_str() << endl;
    string s2("hello world");
    cout << s2.c_str() << endl;
    for (size_t i = 0; i < s2.size(); ++i) {
        s2[i]++;
```
```

```
    }
    cout << s2.c_str()<< endl;//运算符重载的意义
}
```

问题来了，我们能不能使用范围for？范围for的底层实际上是指针，所以我们要解决的实际上是指针的问题。但是迭代器不一定就是指针。
把指针定义一下，这个东西实际上就可以使用了：（详看第五节迭代器）

```
for (auto ch : s2) {//替换成迭代器
    ch++;
    cout << ch ;
}
cout << endl;
string::iterator it1 = s2.begin();
while (it1 != s2.end()) {
    (*it1)++;
    cout << *it1;
    it1++;
}
cout << endl;
```

库里面的size和capacity都是不包含"\0"的
上述方法定义的迭代器在const类型的字符串中是没有办法修改的，原因是会涉及到权限放大的问题，那么我们就要用到函数重载了。（传入的指征是`string const*`的，返回的的是`string*`这样涉及权限放大，所以我们要对模拟迭代器实现一个const重载版本。

```
//typedef const char* const_iterator;
//const_iterator begin() const;
//const_iterator end()const;
void text_string02() {
    const string s2("hello world");//const的话就遍历不了，这里会涉及一个权限放大的问题
    //那为什么会涉及权限放大的问题呢？
    //const string*不能传给string*,但是迭代器本身是需要能修改能遍历的，那这里是不是就不能用了呢？nonono
    //这个时候就体现到函数重载的作用了
    string::const_iterator it1 = s2.begin();
    while (it1 != s2.end()) {
        //(*it1)++;这里就不能修改了
        cout << *it1;
        it1++;
    }
```

```
    cout << endl;
}
```

1.4.2 `oparetor==` 操纵符重载

```
bool string::operator==(const string& s) const {
    //不能复用库里面的实现
    size_t i1 = 0, i2 = 0;
    while (i1 < _size && i2 < s._size) {
        if (_str[i1] != s[i2]) {
            return false;
        }
        else {
            ++i1;
            ++i2;
        }
    }
    return i1==_size&&i2 == s._size;//长度还有相同的嘛
}
```

1.4.3 `oparetor!=` 操纵符重载

```
bool string::operator!=(const string& s) const {
    return !(*this == s);
}
```

1.4.4 `oparetor+=` 操纵符重载

```
string& string::operator+=(char ch) {//加等一个字符
    push_back(ch);
    return *this;
}
string& string::operator+=(const char* str) {//加等一个字符串
    append(str);
    return *this;
}
```

1.4.5 比较运算符重载

1. operator<

```
//比较ASCII值
//1."hello" "hello" ->false
//2."hellox" "hello" ->false
//"hello" "hellox" ->ture
bool string::operator<(const string& s) const{//防止被修改，这里比较本身不需要被修改
    //不能复用库里面的实现
    size_t i1 = 0, i2 = 0;
    while (i1 < _size && i2 < s._size) {//逐一比对
        if (_str[i1] < s[i2]) {
            return true;
        }
        else if (_str[i1] > s[i2]) {
            return false;
        }
        else {
            ++i1;
            ++i2;
        }
    }
    return i2 <s. _size;//这里说明在s与_str长度不一样的时候
    //i2迭代的是s,这个时候i2与i1应该一样大，i2<s._size就说明s比_str长，这个说明_str与s的字符串的前半段相同，但是s比_str长，所以_str要小
}
```

后面的全部复用就好了：

2. operator<=

```
bool string::operator<=(const string& s) const {
    return *this < s || *this == s;
}
```

3. operator>

```
bool string::operator>(const string& s) const {
    //不能复用库里面的实现
    return !(*this < s);
}
```

4. operator<=

```
bool string::operator>=(const string& s) const {
    return *this > s || *this == s;
}
```

1.4.6 流插入运算符

```
//这个读不到空格，为什么
istream& operator>>(istream& in,string& s) {
    //一个字符一个字符的读取直到遇到'\0'
    char ch;
    cin >> ch;//逐一读取进去，但是默认会跳过空格
    while (ch!=' '&&ch!='\n') {
        s += ch;
        in >> ch;//将字符一个一个的读取到ch中
    }
    return in;
}
```

这里会自动忽略掉空格与回车：

```
istream& operator>>(istream& in,string& s) {
    //一个字符一个字符的读取直到遇到'\0'
    char ch = in.get();
    //cin >> ch;因为cin会自动忽略掉空格与回车
    while (ch!=' '&&ch!='\n') {
        s += ch;
        //in >> ch;//将字符一个一个的读取到ch中
        ch = in.get();
    }
    return in;
}//本身就有值不能直接加等
```

让其一个字符一个字符的读入：

```
istream& getline(istream& in, string& s, char delim ) {//自定义分割符来读取字符
串
    //一个字符一个字符的读取直到遇到'\0'
    s.clear();
```

```
    char ch = in.get();
    //cin >> ch;因为cin会自动忽略掉空格与回车
    while (ch != delim) {
        s += ch;
        ch = in.get();
    }
    return in;
}
```

下述代码可能造成空间浪费：当输入一个非常长的串，会不断的扩容，扩容的效率非常低，提前开空间也不知道开多少？

```
istream& operator>>(istream& in,string& s) {
    //一个字符一个字符的读取直到遇到'\0'
    s.clear();
    char ch = in.get();
    //cin >> ch;因为cin会自动忽略掉空格与回车
    while (ch!=' '&&ch!='\n') {

        s += ch;
        //in >> ch;//将字符一个一个的读取到ch中
        ch = in.get();
    }
    return in;
}
```

再优化，让我们可以指定终止条件结束读取字符串，让其不会和库里面的一样遇到`\0`就结束了。

```
istream& getline(istream& in, string& s, char delim) {
    s.clear();
    char ch;
    while (in.get(ch)) {          // 用 in.get(ch) 检查是否成功读取
        if (ch == delim) {
            break;                // 遇到分隔符，停止
        }
        s += ch;
    }
    return in;//直到换行才停止
}
```

如果没有自定义拷贝构造，下述代码就崩溃了，因为没有自定义拷贝构造 → 编译器生成的是浅拷贝 → 两个对象的 `_str` 指向同一块堆内存 → 修改一个会影响另一个，而且析构时会重复释放导致崩溃：

```
void text_string08() {
    string s1("hello"), s2("helloxxxxxxxxx");
    string s3(s1);
    cout << s1 << endl;
    cout << s3 << endl;
    s1[0] = 'x';
    cout << s1 << endl;
    cout << s3 << endl;
}//1.浅拷贝 2.一个修改连带着另一个
```

传值返回的代价：构造临时对象，会构成极大的浪费，但是编译器自己会尝试拷贝构造
赋值重载的逻辑，开一个一样大的空间，再把值拷贝过来
传统写法：自己老老实实开空间，拷贝数据，效率并没有提高，只是写法更加简洁

1.4.7流输出运算符

```
ostream& operator<<(ostream& out,const string& s) {
    out << s.c_str() << endl;
    return out;
}
```

遇见'\0'就都没了,优化

```
//string s2("hello worlk");
//s2 += '\0';
//s2 += '\0';
//s2 += '\0';
//s2 += '*';
//cout << s2 << endl;
ostream& operator<<(ostream& out,const string& s) {
    //out << s.c_str() << endl;
    for (size_t i = 0; i < s.size(); i++) {
        out << s[i];//不要管是啥，直接读取输出出去
    }
    return out;
}
```

还没完

```
//string s2("hello worlk");
s2 += "yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy";
cout << s2 << endl;
//append出现问题,就会对append进行优化
```

1.5 赋值运算符重载

```
string& string::operator=(const string& s) {
    //两个已经存在的对象，防止自己给自己赋值
    if (this != &s) {
        char* tmp = new char[s._capacity + 1];//开空间
        memcpy(tmp, s._str, s._size + 1);
        delete[]_str;
        _str = tmp;
        _size = s._size;
        _capacity = s._capacity;
    }
    return *this;
}
```

赋值重载也可以：复用拷贝构造，让tmp开空间是s1想要的，还会自动清理s1全程没动手

```
string& string::operator=(const string& s) {
    //两个已经存在的对象，防止自己给自己赋值
    if (this != &s) {
        string tmp(s);
        swap(tmp);
    }
    return *this;}
```

还简化：本质是一样的，在传参的时候赋值

```
string& string::operator=(string tmp) {
    swap(tmp);
    return *this;
}
```

现代写法的本质是剥削。让tmp干活

2. 修改器

2.1swap交换函数

直接使用string库里面的内容将成员变量逐一交换

```
void string::swap(string& s) {
    std::swap(_str, s._str);//直接交换内置类型
    std::swap(_size, s._size);
    std::swap(_capacity, s._capacity);
}
```

2.2 push_back()尾插函数

```
void string::push_back(char ch) {//尾插
    //首先判断是否需要扩容
    if (_size >= _capacity) {
        //三目运算符做简单的判断
        size_t newcapacity = _capacity == 0 ? 4 : 2 * _capacity;
        reserve(newcapacity);//但是这里可能出问题，就是capacity走缺省值的话可能是
零
    }
    _str[_size] = ch;
    _size++;
    _str[_size] = '\0';//注意
}
```

2.3 pop_back去掉尾部元素

```
void string::pop_back() {
    assert(_size > 0);//防止越界
    --_size;//减少再补上'\0'
    _str[_size] = '\0';
}
```

2.4 insert插入函数

下面代码有bug，当 `pos == 0` 时，`end` 会循环到 `0`，然后 `--end` 会让它变成 `size_t` 的最大值（例如 18446744073709551615），导致循环无法正常结束，进而发生数组越界访问。

```
void string::insert(size_t pos, char ch) {
    //首先判断是否需要扩容
    if (_size >= _capacity) {
        //三目运算符做简单的判断
        size_t newcapacity = _capacity == 0 ? 4 : 2 * _capacity;
        reserve(newcapacity);
    }
    //数据挪动
    size_t end = _size;
    while (end>=pos) {
        _str[end + 1] = _str[end];
        --end;
    }
    _str[pos] = ch;
    ++_size;
}
```

发生类型提升：

```
void string::insert(size_t pos, char ch) {
    //首先判断是否需要扩容
    if (_size >= _capacity) {
        //三目运算符做简单的判断
        size_t newcapacity = _capacity == 0 ? 4 : 2 * _capacity;
        reserve(newcapacity);
    }
    //数据挪动
    int end = _size;

    while (end>=(int)pos) {//end向pos提升
        _str[end + 1] = _str[end];
        --end;
    }
    _str[pos] = ch;
    ++_size;
}
```

优化：挪动范围只覆盖 `[pos+1, _size+1]`，而且循环条件是 `end > pos`，永远不会出现无符号回绕问题。

```
void string::insert(size_t pos, char ch) {
    //首先判断是否需要扩容
    if (_size >= _capacity) {
        //三目运算符做简单的判断
        size_t newcapacity = _capacity == 0 ? 4 : 2 * _capacity;
        reserve(newcapacity);//但是这里可能出问题，就是capacity走缺省值的话可能是
零
    }
    //数据挪动
    size_t end = _size + 1;
    while (end>pos) {//end向pos提升
        _str[end] = _str[end-1];//前面的向后放
        --end;
    }
    _str[pos] = ch;
    ++_size;
}
```

同样的插入一个串：

```
void string::insert(size_t pos, const char* str) {
    assert(pos <= size());
    size_t len = strlen(str);
    if (_size + len >= _capacity) {
        size_t newcapacity = _capacity == 0 ? 4 : _capacity * 2;
        while (newcapacity < _size + len) {   // 确保够大
            newcapacity *= 2;
        }
        reserve(newcapacity);
    }
    int end = _size;
    while (end >= (int)pos) {
        _str[end + len] = _str[end];
        end--;
    }
    for(size_t i= 0; i < len; i++) {
        _str[pos + i] = str[i];
    }
    _size += len;

}
```

优化：

```
void string::insert(size_t pos, const char* str) {
    assert(pos <= size());
    size_t len = strlen(str);
    if (_size + len >= _capacity) {
        size_t newcapacity = _capacity == 0 ? 4 : _capacity * 2;
        while (newcapacity < _size + len) {   // 确保够大
            newcapacity *= 2;
        }
        reserve(newcapacity);
    }
    size_t end = _size+len;
    //while (end >pos) {越界
    while (end > pos+len-1) {//最后一次落在的位置
        _str[end] = _str[end-len];
        end--;
    }
    for(size_t i= 0; i < len; i++) {
        _str[pos + i] = str[i];
    }
    _size += len;

}
```

2.5 erase修改函数

定义和声明npos:
声明npos，npos是一个哨兵值，`size_t` 是**无符号整数**（`unsigned`），`-1` 转换成无符号整数后，会变成该类型的**最大值**（例如 `18446744073709551615`）。这个值比任何合法的下标（`0 ~ _size-1` 或 `_size`）都大，因此，它非常安全地表示 **“不是任何一个有效位置”**
同时，任何长度 `len` 都不会超过 `npos`（除非你手动传了一个超大值）

```
public:
    static const size_t npos;
```

定义npos:

```
const size_t string::npos = -1;//指定类域
```

具体代码示例：

```
void string::erase(size_t pos, size_t len) {
    assert(pos < _size);
    //要删除的数据 ，大于pos后面的字符个数
    //pos后面全部删除
    if (len == npos || len >= (_size - pos))
    {
        _size = pos;
        _str[_size] = '\0';
    }
    else
    {
        size_t i = pos + len;//重新开辟一个空间将数据拷贝进去
        memmove(_str + pos, _str + i, _size + 1 - i);
        _size -= len;
    }
}
```

2.5 clear清除

```
void string::clear() {
    //不释放空间
    _str[0] = '\0';
    _size = 0;
}
```

3. 容量和空间相关

3.1reserve空间开辟函数

```
void string::reserve(size_t n) {
    if (n > _capacity) {//给外部使用时防止缩容
        //手动开辟空间
        char* str= new char[n+1];//手动开辟内存,为'\0'预留出一块空间
        memcpy(str, _str,_size+1);
        delete[] _str;
        _str = str;
        _capacity = n;
    }
}
```

3.2 append 追加函数

```
void string::append(const char* str) {
    //先算一下
    size_t len = strlen(str);
    if (_size + len > _capacity)//这里判断是否需要扩容
    {
        size_t newcapacity = size+len>2*capacity  ? 2 * _capacity:_size+len;
        reserve(newcapacity);
    }
    strcpy(_str + _size, str);//区别？手动测算'\0'并避免写两次
    _size += len;
}
```

修正上述函数：

```
void string::append(const char* str) {
    //先算一下
    size_t len = strlen(str);
    if (_size + len > _capacity)
    {
        size_t newcapacity = size+len>2*capacity  ? 2 * _capacity:_size+len;
        reserve(newcapacity);
    }
    //strcpy的问题
    memcpy(_str + _size, str,len+1);//区别？手动测算'\0'并避免写两次
    _size += len;
}
```

3.3 size返回大小

```
//size_t size() const;
size_t string::size() const {
    return _size;
}
```

4.查找和提取

4.1c_str返回常量指针

```
const char* string::c_str() const
{//c_str返回char*,可以用char*直接完成打印
    return _str;//在这里返回_str如果直接输出打印的话
}
```

4.2 find查找函数

查找某个字符：

```
size_t string::find(char ch, size_t pos)const {
    for (size_t i = pos; i < _size; i++) {
        if (_str[i] == ch) {
            return i;
        }
    }
    return npos;
}
```

查找某个子串：

```
size_t string::find(const char* str, size_t pos)const{//查找子串
    //1.暴力查找strstr
    //2.KMP
    const char* p1 = strstr(_str + pos, str);
    if (p1 == nullptr) {
        return npos;
    }
    else {
        return p1 - _str;//步长
    }

}
```

4.3 substr提取某一段元素

```
string string::substr(size_t pos, size_t len) const//传值返回，返回的时临时对象
{
    if (len == npos || len >= _size - pos) {
        len = _size - pos;
    }
```

```
    string ret;
    ret.reserve(len);
    for (size_t i = 0; i < len; i++) {
        ret += _str[pos + i];//不断追加
    }
    //cout << &ret << endl;
    return ret;
}
```

在这里，我们可以写一个网站的域名提取的函数：

```
void split_url(const string& url) {//临时对象的传入需要调用拷贝构造，但是我们没有写拷贝构造
    //浅拷贝要使用的时候就已经析构了
    size_t i1 = url.find(':');
    if (i1 != string::npos)
    {
        string ret = url.substr(0, i1);
        cout << &ret << endl;
        cout << ret << endl;
    }
    size_t i2 = i1 + 3;
    size_t i3 = url.find('/', i2);
    if (i3 != string::npos) {
        cout << url.substr(i2, i3 - i2)<<endl;
        cout << url.substr(i3 + 1)<<endl;
    }
    cout << endl;
}
void text_string05() {
    string url1 = "https://chat.deepseek.com/a/chat/s/bb7f967c-211e-4e24-a838-b6f54cd5750b";
    string url2 = "https://edu.bitejiuyeke.com/login";
    split_url(url1);
    split_url(url2);
}
```

5.迭代器

我们需要在自定义的类型中使用迭代器，我们应该如何做呢？迭代器底层的本质其实是指针，我们可以自己定义一下：

```
//声明
typedef char* iterator;
iterator begin();
iterator end();
//重载一份
typedef const char* const_iterator;
const_iterator begin() const;//这个不是迭代器本身不可修改，而是让迭代器指向的内容不可修改
const_iterator end()const;
//定义
string::iterator string::begin() {
    return _str;//返回的是第一个元素的地址
}
string::iterator string::end() {
    return _str + _size;//返回的是最后一个元素的地址的地址
}
string::const_iterator string::end()const {
    return _str + _size;//指向的的值不能修改
}
string::const_iterator string::begin()const {
    return _str;
}
```